Solving Job-Shop Scheduling via Search, Heuristics, and Local Search

Víctor Puga
a01568636@tec.mx

Eduardo Porto
a01027893@tec.mx

October 31, 2026

Abstract

We model the job-shop scheduling problem (JSP) as a search problem and compare depth-first search with pruning, A* with two admissible heuristics, and multi-start hill climbing with two neighborhoods, all with a 60 second limit per run. The exact methods only solve small instances: A* solves `ft06` (6×6) but times out on `la01`, `ft10` and `la36`. Local search is fast and always returns a schedule, but it rarely reaches the optimum and its quality drops as instances grow. The same pattern appears on 20 generated instances, where the exact methods solve the easy ones and only some of the medium ones.

1 Introduction

The Job-Shop Scheduling Problem (JSP) is a classic, NP-hard [4] optimization problem where jobs have to be processed on machines. The goal is to minimize the time when the last operation ends, also called makespan. We model it as a search problem and compare uninformed search (DFS), heuristic search (A* with two heuristics) and local search (multi-start hill climbing, two neighborhoods). Each algorithm has a maximum time of 60 seconds per instance to find the solution with the lowest makespan it can.

2 Problem Modeling

2.1 Problem Definition

There are n jobs and m machines. Each job j has a fixed sequence of m operations $O_{j,1}, O_{j,2}, \ldots, O_{j,m}$. Operation $O_{j,k}$ runs on machine $\mu(j,k)$ for $p_{j,k}$ time units. A schedule gives every operation a start time $s_{j,k} \geq 0$ and must satisfy three constraints:

- **Precedence:** an operation cannot start until the previous operation of its job has finished ($s_{j,k+1} \geq s_{j,k} + p_{j,k}$).
- **Machine capacity:** a machine processes at most one operation at a time.
- **No preemption:** once an operation starts, it runs until it ends.

The objective is to minimize the makespan $C_{\max} = \max_{j,k}(s_{j,k} + p_{j,k})$, the time at which the last operation ends. In the code an instance is two matrices, `machines[job, operation]` and `durations[job, operation]`.

2.2 Uninformed Search Formulation

For DFS, we build the schedule one operation at a time. A state is a partial schedule: the next operation of each job, the time each job is free, and the time each machine is free. In the initial state everything is free at time 0. An action dispatches the next operation of a job that is not finished. The operation starts as early as possible, at max(job free, machine free), so every schedule we build is semi-active. The path cost $g(s)$ is the makespan of the partial schedule. Branches are pruned with a lower bound against the best solution found so far.

The search space is large. The number of dispatch sequences is $(nm)!/(m!)^n$, which is 369 600 for a 4×3 instance and about 2.7×10^{24} for a 6×6 one.

2.3 Informed Search Formulation

A* [5] uses the same states, actions and transitions as DFS. It adds a heuristic $h(s)$ that estimates the remaining cost, and expands the state with the lowest $f(s) = g(s) + w \cdot h(s)$. With $w = 1$ this is plain A*, and $w > 1$ gives Weighted A*. Our heuristics are lower bounds built from job workloads and machine loads (section 3.2).

2.4 Local Search Formulation

Local search skips the tree and works on complete solutions. A solution is a full dispatch sequence, and decoding it gives a valid schedule, so no repair is needed. Its cost is the makespan of the decoded schedule. A neighbor is a small change to the sequence, like swapping two elements. Neighbors keep the same number of copies of each job, so they are always feasible. We lose the optimality guarantee, but it scales to larger instances.

3 Methods

3.1 Uninformed Search Baseline

To provide a baseline, we implement a depth-first search algorithm (DFS) with branch and bound. It is not the most efficient uninformed search, but it is simple and fast enough to solve the small instances. It also shares its lower bound with H1, so it isolates the effect of the search order. The lower bound $LB(s)$ is the maximum of three values: the makespan so far, the free time of a job plus its remaining durations (over all jobs), and the free time of a machine plus the remaining durations on it (over all machines). Computing $LB(s)$ costs $O(nm)$.

The algorithm considers a goal state when all operations have been dispatched (depth nm), and it keeps track of the best makespan C^* found so far. We prune a branch when $LB(s) \geq C^*$. No completion of s finishes before $LB(s)$, so pruning never discards a schedule better than C^*. If the search finishes within the time budget, the best makespan found is optimal. Otherwise it is only an upper bound.

3.2 Heuristic Search

We use A* to improve upon the DFS baseline. It uses the same states and transitions as DFS, but it expands them in order of $f = g + h$, where g is the cost so far (makespan of the partial schedule) and h is a heuristic estimate of the remaining cost. We implement two heuristics, H1 and H2, to compare a cheap but less informative heuristic with a more expensive but more informative one. A* is optimal when h never overestimates the remaining cost, and both heuristics are admissible. To avoid re-expanding duplicates, a state is identified by its key: the next operation of each job, the job free times and the machine free times. We skip states whose key was already popped.

H1 (cheap). The first heuristic is the lower bound used in the DFS baseline: $h_1(s) = LB(s) - g(s)$. It is cheap because one pass over the unscheduled operations is enough: $O(nm)$ per state.

H2 (informative). The second heuristic is a more informative lower bound, $LB_2(s)$, based on the one-machine bound, like the head/tail bounds in the work of Carlier [2]. R_k is the set of operations yet to run on machine k. For each operation, we compute its head (earliest start) and tail (duration of the operations after it in its job). For each machine, we compute the smallest head, plus the sum of durations in R_k, plus the smallest tail. $LB_2(s)$ is the maximum of that value over all machines and $LB(s)$, and $h_2(s) = LB_2(s) - g(s)$. This makes H2 admissible and never below H1. Its cost is also $O(nm)$ per state, but with a bigger constant: on `ft06` and the larger instances H2 expands 2.5 to 3.4 times fewer states per second than H1 (table 4), so each expansion costs about that much more.

3.3 Local Search

A* and DFS cannot finish on the larger instances within 60 seconds, so we add local search to get good schedules fast. We implement multi-start hill climbing with first improvement: it starts from a random solution and moves to a neighbor with a lower makespan until none is found, then restarts.

The representation is the dispatch sequence, a list with each job repeated once per operation (a permutation with repetition). The k-th occurrence of a job is its k-th operation, so every sequence decodes to a feasible semi-active schedule and its makespan is its cost. We compare two neighborhoods over this representation. `swap` exchanges two random positions, and `insert` removes the job at one random position and reinserts it at another, shifting the ones in between. Each iteration samples up to 25 neighbors and moves to the first one that lowers the makespan. The cost per neighbor is $O(nm)$ because of the decoding.

Each restart begins from a random shuffled sequence, and it stops after 500 iterations or when none of the 25 sampled neighbors improves. The defaults are 10 restarts and seed 42, and each seed has its own random generator. The search is bounded by restarts × iterations, not by the 60 s budget.

4 Experimental Setup

4.1 Instances

We use a mix of standard benchmarks, our own small instances, and a seeded generator for larger instances. Benchmarks come from the OR-Library [1]. We use the Fisher–Thompson instances `ft06` and `ft10` [3], and the Lawrence instances `la01` and `la36` [6]. For our own testing, we create a 4×3 example and a 4×4 cut from `ft06` (first 4 jobs, machines 0–3). Its optimum, 33, was computed with our DFS. It is not an official benchmark. The details are summarized in table 1.

We also generate 20 random instances with a seeded generator. It creates a random permutation of the machines for each job, and random durations in a given range. Instance number k (from 0, in the order easy, medium, hard) uses the seed $2026 + k$, so the whole set can be regenerated with `scripts/generate_instances.py`. Generated instances have no known optimum, so their gaps are measured against the best makespan found in any run. The instances are grouped into three families: `gen_easy`, `gen_medium` and `gen_hard`. Each family has a different number of instances (table 2).

Table 1: Instances used in the experiments.

Instance	Jobs × Machines	Durations	Optimum	Source
`example`	4×3	1–10	29	ours
`ft06_4x4`	4×4	1–10	33	cut from `ft06`
`ft06`	6×6	1–10	55	Fisher–Thompson
`la01`	10×5	12–98	666	Lawrence
`ft10`	10×10	2–99	930	Fisher–Thompson
`la36`	15×15	7–99	1268	Lawrence

Table 2: Generated instances.

Family	Count	Jobs × Machines	Durations
`gen_easy_01–10`	10	3×3 or 4×4	1–10
`gen_medium_01–08`	8	5×5, 6×6 or 7×7	1–50
`gen_hard_01–02`	2	10×10, 15×10	1–99

4.2 Protocol and Metrics

Each problem instance is run on every algorithm with a maximum time limit of 60 seconds. If an algorithm does not finish within this time, it is terminated externally and considered a timeout. A killed run has no `done` event in its logs, so we take the last logged event to report the best makespan found, the number of visited states, and the time to reach that best solution.

All runs use single-threaded Python 3.12.15 on macOS 27.2 with an M1 chip.

For every run we report the final makespan, the time to best, the number of visited states, the maximum frontier size (A* only), and the gap to the known optimum. A visited state is one call to `search` in DFS, a popped non-duplicate state in A*, and an evaluated sequence in local search.

DFS and A* are deterministic, so each case has a single run. Local search is stochastic, so we run it with seeds 1 to 10, one run per seed, for each neighborhood. Its success rate is the fraction of seeds that reach the known optimum, and we report the minimum, mean and sample standard deviation ($n = 10$) of the final makespan. Local search stops after 10 restarts of at most 500 iterations each, so it normally finishes well before the 60 second budget. The ablations compare H1 against H2 in A*, `swap` against `insert` in local search, and 10 restarts against 1 restart.

Outside the comparison, we also ran pruned DFS on `ft06` for 1 hour and A* with H1 on `la01` with no time limit (stopped after 44 minutes). They are only used in the discussion.

4.3 Reproducibility

The code is in the `job-shop-problem` repository. Its README explains the setup and the command-line options of `main.py`. The script `scripts/benchmark.sh` runs the benchmarks, with `TIMEOUT=60` by default. Each run writes a JSON Lines log `logs/<algorithm>_<instance>_<tag>_<timestamp>.jsonl`. The `start` event records the options, Python version, platform and git commit. The `done` event records the result, `time_to_best`, `max_frontier` (A*) and whether the known optimum was reached.

The logs of this report are in `results/final_benchmarks/` and `results/generated_instances_benchmarks/` of that repository. All tables, figures and CSV summaries come from these logs with `scripts/analyze_results.py`. Unit tests are located in `tests/`.

5 Results

Standard instance results come from 157 runs and generated instance results from 460 runs, all with the 60 s limit. T/O means the run was killed at 60 s.

5.1 DFS: Pruning vs. No Pruning

Pruning cuts the visits 24 times on `example` and makes it 12 times faster (table 3). On `ft06_4x4` the difference is bigger: pruned DFS finishes in 3.0 s, while the unpruned one times out. The unpruned run had already found the optimum (33) at 10 s, but it cannot prove it without exploring everything. On `ft06`, pruned DFS times out with a makespan of 96 (74.5% gap) that it found at 10.3 s and never improved. We did not run DFS on `la01`, `ft10` and `la36`, since it already fails on `ft06`.

Table 3: DFS with and without pruning. For T/O runs the makespan is the best found, and visits and time are from the last logged event.

Instance	Prune	Status	Makespan	Gap (%)	Visits	Time (s)	To best (s)
`example`	off	done	29	0.0	1 107 697	1.86	0.094
`example`	on	done	29	0.0	45 736	0.15	0.022
`ft06_4x4`	off	T/O	33	0.0	36 200 000	60	9.979
`ft06_4x4`	on	done	33	0.0	858 401	3.04	1.088
`ft06`	on	T/O	96	74.5	12 900 000	60	10.321

5.2 A*: H1 vs. H2

Both heuristics solve `example`, `ft06_4x4` and `ft06` optimally and time out on `la01`, `ft10` and `la36` (table 4). H2 expands between a third and two thirds fewer states on the three solved instances, and on `ft06` its

frontier is 3 times smaller. The time gain is smaller, though: on `ft06` H2 takes 26.0 s and H1 31.9 s, because each H2 expansion is about 2.5 times slower. On the two smallest instances both finish in a few milliseconds, so their order does not mean anything.

For the timed-out runs, the last popped f is a lower bound on the optimum. H2 reaches higher bounds on `ft10` (796 vs. 655) and `la36` (1 098 vs. 1 028). On `la01` both reach 666, which is the optimum, but the goal state was not popped before the timeout. Compared with pruned DFS, A* with H1 needs 815 expansions instead of 858 401 visits on `ft06_4x4`, and it finds the optimum on `ft06` where DFS does not.

Table 4: A* with H1 (`lower_bound`) and H2 (`machine_heads_tails`). Frontier is the maximum size. Last f is the f of the last progress event of a timed-out run.

Instance	Heur.	Status	Makespan	Last f	Expanded	Frontier	Time (s)	Exp./s
`example`	H1	done	29	–	447	598	0.008	55 875
`example`	H2	done	29	–	290	374	0.011	26 364
`ft06_4x4`	H1	done	33	–	815	1 159	0.109	7 477
`ft06_4x4`	H2	done	33	–	270	461	0.015	18 000
`ft06`	H1	done	55	–	509 559	1 627 247	31.912	15 968
`ft06`	H2	done	55	–	163 290	532 295	26.006	6 279
`la01`	H1	T/O	–	666	405 360	2 940 580	60	6 767
`la01`	H2	T/O	–	666	130 790	960 170	60	2 183
`ft10`	H1	T/O	–	655	226 610	1 775 743	60	3 783
`ft10`	H2	T/O	–	796	68 120	537 927	60	1 137
`la36`	H1	T/O	–	1 028	71 830	824 363	60	1 199
`la36`	H2	T/O	–	1 098	20 960	247 206	60	350

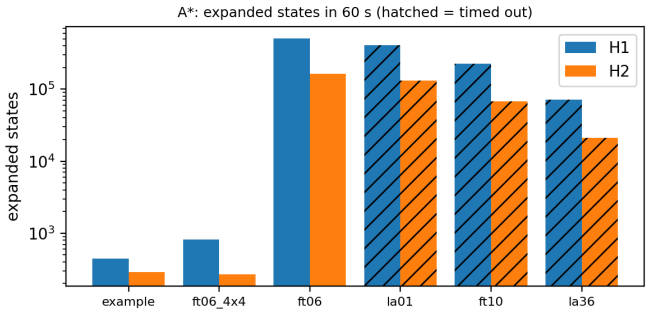


Figure 1: Expanded states of A* with H1 and H2 (log scale).

5.3 Local Search

We ran multi-start hill climbing with seeds 1 to 10 and at most 500 iterations per restart (table 5, fig. 2). It reaches the optimum on `example` in all 10 seeds, but never on the other instances. With `swap` and 10 restarts the mean gap grows with instance size, from 6.5% on `ft06` and 7.2% on `la01` to 32.0% on `ft10` and 37.7% on `la36`.

Between the neighborhoods (`swap` vs. `insert`) there is no clear winner. `insert` has the better mean on `ft10` and `swap` on the other three, but the differences are small compared to the standard deviation (25.9 and 58.8 on `ft10`). Restarts matter more: 10 restarts beat 1 restart on every instance, for example the gap on `la36` goes from 52.1% to 37.7% and the standard deviation also drops. This is not an equal-budget comparison, since 10 restarts use 10 times more evaluations. Every run ends in under 0.42 s, so the 60 s budget is never used.

Table 5: Local search over 10 seeds. Opt. is the known optimum, Hits the number of seeds that reached it, Time the mean run time.

Instance	Config	Opt.	Min	Mean	Std	Hits	Gap mean (%)	Time (s)	To best (s)
`example`	insert, 10 restarts	29	29	29.0	0.0	10/10	0.0	0.006	0.001
`example`	swap, 10 restarts	29	29	29.0	0.0	10/10	0.0	0.006	0.000
`ft06`	insert, 10 restarts	55	58	59.6	1.6	0/10	8.4	0.024	0.011
`ft06`	swap, 1 restart	55	59	64.9	6.2	0/10	18.0	0.003	0.002
`ft06`	swap, 10 restarts	55	57	58.6	1.1	0/10	6.5	0.025	0.013
`la01`	insert, 10 restarts	666	678	720.9	28.1	0/10	8.2	0.045	0.025
`la01`	swap, 1 restart	666	687	766.2	59.2	0/10	15.0	0.005	0.003
`la01`	swap, 10 restarts	666	687	713.8	15.9	0/10	7.2	0.051	0.019
`ft10`	insert, 10 restarts	930	1 088	1197.8	58.8	0/10	28.8	0.123	0.062
`ft10`	swap, 1 restart	930	1 246	1349.1	71.9	0/10	45.1	0.012	0.008
`ft10`	swap, 10 restarts	930	1 172	1227.8	25.9	0/10	32.0	0.116	0.074
`la36`	insert, 10 restarts	1268	1 737	1780.4	23.7	0/10	40.4	0.342	0.216
`la36`	swap, 1 restart	1268	1 721	1929.2	148.3	0/10	52.1	0.033	0.026
`la36`	swap, 10 restarts	1268	1 648	1745.6	66.0	0/10	37.7	0.318	0.194

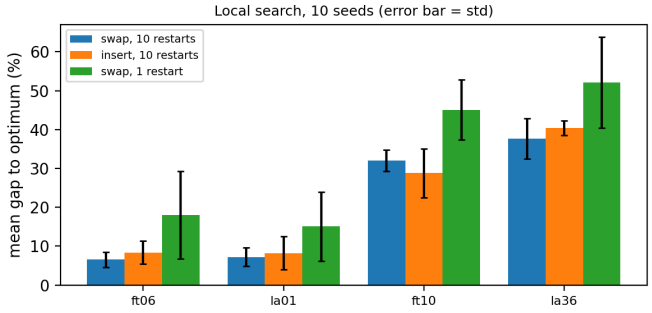


Figure 2: Mean gap to the optimum of the three local search configurations.

5.4 Generated Instances

The generated instances have no known optimum, so we use the best makespan found by any run as reference. That reference is proven optimal only where DFS or A* finished, which are the 10 easy instances and four medium ones (`gen_medium_01`, `03`, `04` and `06`). For the rest it comes from local search, so those gaps can be optimistic.

DFS solves the 10 easy instances and times out on all medium and hard ones, with gaps from 43.6% to 163.8% on medium and about 300% on hard. A* with either heuristic also solves the easy instances. On medium, H1 solves 3 of 8 and H2 solves 4, because H2 also finishes `gen_medium_01` (in 37.7 s). Neither solves the hard ones.

H2 expands fewer states on all easy instances and on the 3 medium instances that both solve, but it is not faster on those 3 (for example 45.6 vs. 24.5 s on `gen_medium_04`). So the time gain seen on `ft06` does not hold here. On the timed-out medium instances H2 gives a higher last f on three and the same value on the rest. Figure 3 shows the expanded states of both heuristics.

Local search ends in under 0.28 s. It reaches the reference in 91 of 100 runs on easy, 17 of 80 on medium and 2 of 20 on hard instances with `swap`, and 89, 19 and 0 with `insert`. The neighborhoods are again too close to tell apart. On the hard instances A* only gives lower bounds (665 and 984), so the best local search results (936 and 1 271) are at most 41% and 30% above the optimum.

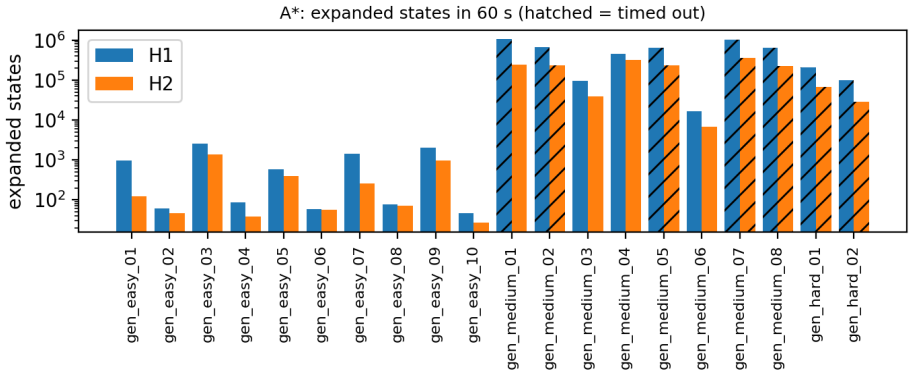


Figure 3: Expanded states of A* with H1 and H2 on the generated instances (log scale, hatched = timed out).

6 Discussion

6.1 Scalability of DFS and A*

DFS did not get far. Without pruning it cannot finish even the 4×4 cut in 60 s, and with pruning it solves the cut but not `ft06`. We think the reason is the visiting order: DFS tries jobs by index, so its first schedules are bad (152 on `ft06`, and only 96 after 10 s) and the bound cannot prune much early on. More time does not fix it, since a 1 hour run on `ft06` (817 million visits) still had the makespan of 96 found at 10 s, against an optimum of 55.

A* solves `ft06` in 26 s with H2, but it times out on `la01` (10×5), `ft10` and `la36` with both heuristics. So in our Python implementation the limit is somewhere between 6×6 and 10×5. Memory is also a problem: on `ft06` with H1 the frontier reaches 1.6 million states, and on `la01` it reaches 2.9 million at the timeout. The 60 s limit is not the only reason A* fails on `la01`. We ran H1 without a limit and stopped it after 44 minutes, with 7.3 million expansions and a frontier of 51.7 million states. The f value was already 666 (the optimum) at the start, but the goal was never popped. It also slowed down over time (the seventh million expansions took 15 minutes against 2.5 for the first), probably from memory pressure.

6.2 H1 vs. H2

H2 is more informed (higher final f-values), but each state costs 2.5 to 3.4 times more. That pays off on `ft06`, where H2 finishes about a fifth faster. We did not expect the gain to disappear on the generated instances: on the medium ones that both heuristics solve, H2 is slower despite fewer expansions. It only pays off on `gen_medium_01`, which H1 does not solve in 60 s. We only ran the long experiment with H1, so we cannot say whether H2 would win on instances where neither finishes.

6.3 Local Search

Local search is fast, but the quality drops and the variance between seeds grows on bigger instances. On `la36` the exact methods give no schedule at all, while local search gives one in 0.3 s that is 38% above the optimum. Each restart stops after very few iterations (2 to 29 on `la36`), so most of the 60 s budget is unused. More restarts would probably help, but we did not try it.

6.4 Limitations

DFS and A* have no internal timeout, so timed-out runs were killed from outside and their state counts come from the last logged checkpoint. Local search stops by iteration limit and not by clock, so it is not an equal-time comparison with DFS and A*, and its stopping rule (25 neighbors without improvement) does not prove a local

optimum. Finally, local search has 10 seeds per instance but DFS and A* are single runs, so their times may vary a bit.

7 Conclusion

Pruned DFS and A* find optimal schedules only on small instances, and A* is the better of the two since it solves `ft06`. H2 expands fewer states than H1, but this rarely saves time because each expansion is more expensive. Local search is the only method that returns a schedule on every standard instance within 60 s, at the cost of no optimality guarantee. Among its options, restarts helped more than the choice of neighborhood.

AI-use Statement

Tools used: Claude Code v2.1.311, using model Claude Sonnet 5.5.
What we used them for:

- Verify and enhance the code we wrote.
- Design unit tests.
- Check the structure of the report.

What was designed by our team:

- State/action/cost modeling
- Heuristics H1/H2 design
- Experimental protocol (seeds, 60 s limit, instances)
- Local search representation and neighborhoods

How we verified correctness: We wrote 51 unit tests, all passing, including a hand-solved 2×2 instance, a brute-force solver on random tiny instances, and checks that both heuristics are admissible. A* also matches the known optima on `example`, `ft06_4x4` and `ft06`. All results come from the logs of `scripts/benchmark.sh`, aggregated by `scripts/analyze_results.py`.
Parts we could not defend without external help: None.

References

[1] J. E. Beasley. OR-Library: Distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11):1069–1072, 1990.

[2] J. Carlier. The one-machine sequencing problem. *European Journal of Operational Research*, 11(1):42–47, 1982.

[3] H. Fisher and G. L. Thompson. Probabilistic learning combinations of local job-shop scheduling rules. In J. F. Muth and G. L. Thompson, editors, *Industrial Scheduling*, pages 225–251. Prentice-Hall, Englewood Cliffs, NJ, 1963.

[4] M. R. Garey, D. S. Johnson, and Ravi Sethi. The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2):117–129, 1976.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[6] S. Lawrence. Resource constrained project scheduling: An experimental investigation of heuristic scheduling techniques (supplement). Technical report, Graduate School of Industrial Administration, Carnegie Mellon University, Pittsburgh, PA, 1984.